

咖啡管理系統 (Coffee) - 教學文件

本文件的設計 (JAX-RS + JPA 三層式架構) 整理而成，並套用到 (Coffee Entity) 以建立一個完整、可擴充的咖啡資料管理 API 教學。

學習目標

- 了解如何將 JPA 實體 (Coffee) 擴展為完整的 CRUD REST API
 - 熟悉三層式架構：Controller (Service) / DAO / Entity
 - 能夠快速建立 Coffee 模組
 - 學會撰寫 API 規格、實作與測試資料
-

架構與檔案定位

Coffee 模組：

- Controller 層：CoffeeService (JAX-RS REST)
 - DAO 層：CoffeeDAO (JPA CRUD)
 - Entity 層：Coffee (已存在於 coffee.txt)
-

既有 Coffee 實體重點

```
@Entity
@Table(name="coffees")
@NamedQuery(name="Coffee.findAll", query="SELECT c FROM Coffee c")
public class Coffee implements Serializable {
    @Id
    @Column(name="COF_NAME")
    private String cofName; // 主鍵：咖啡名稱

    private BigDecimal price; // 價格
    private int sales;        // 銷售量

    @Column(name="SUP_ID")
    private int supId;        // 供應商 ID (對應 Supplier.supId)

    private int total;        // 總量或庫存
}
```

注意事項：

- 主鍵是字串 cofName，因此 API Path Param 應使用字串而不是數字。
 - 目前 supId 僅為數字欄位，若要與 Supplier 關聯，建議改為 @ManyToOne (見「改進建議」)。
-

DAO 層實作 (範例)

以下示範 `CoffeeDAO` 的典型 CRUD :

```
package model;

import javax.persistence.*;
import java.util.*;

public class CoffeeDAO {
    private EntityManager createConnection() {
        try {
            EntityManagerFactory factory =
                Persistence.createEntityManagerFactory("mvjpa1104");
            return factory.createEntityManager();
        } catch (Exception ex) {
            System.out.println("connection error " + ex.getMessage());
            return null;
        }
    }

    public List<Coffee> getAll() {
        EntityManager mgr = createConnection();
        TypedQuery<Coffee> query = mgr.createNamedQuery("Coffee.findAll",
            Coffee.class);
        return query.getResultList();
    }

    public Coffee findByName(String cofName) {
        EntityManager mgr = createConnection();
        Coffee found = mgr.find(Coffee.class, cofName);
        return found; // 若找不到回傳 null
    }

    public boolean addCoffee(Coffee c1) {
        EntityManager mgr = createConnection();
        mgr.getTransaction().begin();
        try {
            mgr.persist(c1);
            mgr.getTransaction().commit();
            return true;
        } catch (Exception ex) {
            System.out.println("add coffee error " + ex.getMessage());
            mgr.getTransaction().rollback();
            return false;
        }
    }

    public boolean updateCoffee(String cofName, Coffee c1) {
        EntityManager mgr = createConnection();
        Coffee found = mgr.find(Coffee.class, cofName);
        if (found == null) return false;
    }
}
```

```
        mgr.getTransaction().begin();
        try {
            // 確保主鍵一致
            c1.setCofName(cofName);
            mgr.merge(c1);
            mgr.getTransaction().commit();
            return true;
        } catch (Exception ex) {
            System.out.println("update coffee error " + ex.getMessage());
            mgr.getTransaction().rollback();
            return false;
        }
    }

    public boolean deleteCoffee(String cofName) {
        EntityManager mgr = createConnection();
        Coffee found = mgr.find(Coffee.class, cofName);
        if (found == null) return false;
        mgr.getTransaction().begin();
        try {
            mgr.remove(found);
            mgr.getTransaction().commit();
            return true;
        } catch (Exception ex) {
            System.out.println("delete coffee error " + ex.getMessage());
            mgr.getTransaction().rollback();
            return false;
        }
    }
}
```

Controller 層實作 (REST API 範例)

`cofName` 為主鍵，因此 Path 參數使用字串：

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import java.util.*;
import model.*;

@Path("/coffees")
public class CoffeeService {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Coffee> getAll() {
        CoffeeDAO dao = new CoffeeDAO();
        return dao.getAll();
    }
}
```

```
@GET
@Path("/{name}")
@Produces(MediaType.APPLICATION_JSON)
public Response findByName(@PathParam("name") String name) {
    CoffeeDAO dao = new CoffeeDAO();
    Coffee c1 = dao.findByName(name);
    if (c1 != null) {
        return Response.status(Response.Status.OK).entity(c1).build();
    } else {
        return Response.status(Response.Status.NOT_FOUND).build();
    }
}

@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response addCoffee(Coffee c1) {
    CoffeeDAO dao = new CoffeeDAO();
    boolean ok = dao.addCoffee(c1);
    return ok ? Response.ok(c1).build() :
Response.status(Response.Status.BAD_REQUEST).build();
}

@PUT
@Path("/{name}")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response updateCoffee(@PathParam("name") String name, Coffee c1) {
    CoffeeDAO dao = new CoffeeDAO();
    boolean ok = dao.updateCoffee(name, c1);
    return ok ? Response.ok(c1).build() :
Response.status(Response.Status.BAD_REQUEST).build();
}

@DELETE
@Path("/delete/{name}")
@Produces(MediaType.APPLICATION_JSON)
public Response deleteCoffee(@PathParam("name") String name) {
    CoffeeDAO dao = new CoffeeDAO();
    Coffee c1 = dao.findByName(name);
    boolean ok = dao.deleteCoffee(name);
    return ok ? Response.ok(c1).build() :
Response.status(Response.Status.NOT_FOUND).build();
}
}
```

API 規格與範例

- 取得全部：GET /coffees
- 取得單筆：GET /coffees/{name} 例如 /coffees/Colombian
- 新增：POST /coffees

- 更新：PUT /coffees/{name}
- 刪除：DELETE /coffees/delete/{name}

範例 JSON：

```
{
  "cofName": "Colombian",
  "price": 5.50,
  "sales": 1200,
  "supId": 101,
  "total": 5000
}
```

建置與設定 (與 Supplier 模組共用)

- persistence.xml：使用相同的 persistence-unit (例如 mvjpa1104)
- DB 表：coffees (已由 Entity 對應)。可在 MySQL 先建立：

```
CREATE TABLE coffees (
  COF_NAME varchar(50) PRIMARY KEY,
  PRICE decimal(10,2),
  SALES int,
  SUP_ID int,
  TOTAL int
);
```

與 Supplier 的關聯 (進階)

目前是以 supId 數值欄位儲存。若要以 ORM 關聯方式表達：

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "SUP_ID", insertable = false, updatable = false)
private Supplier supplier;
```

備註：

- 若改成真正外鍵關聯，建議將 supId 欄位改為由 JPA 維護 (insertable = true, updatable = true)，並在傳入 JSON 時以 supplier 物件或 supId 進行轉換。

改進建議 (針對 coffee.txt 與整體模組)

1. 加上 Bean Validation (避免不合理資料)：
 - @NotBlank 用於 cofName

- `@DecimalMin("0.0")` 用於 `price`
 - `@Min(0)` 用於 `sales`、`total`
 - 2. 設計命名查詢：
 - 依價格區間、供應商查詢：`Coffee.findBySupplier`, `Coffee.findByPriceBetween`
 - 3. DAO 連線管理：
 - 抽取共用的 `EntityManagerProvider`，避免重複建立 `EntityManagerFactory`
 - 4. API 回應統一格式：
 - 以回應模型包裝，例如 `{ data, message, success }`
 - 5. 錯誤處理與日誌：
 - 使用 `try-catch` + 統一例外對應 (JAX-RS `ExceptionHandler`)
 - 6. 分頁與排序：
 - `GET /coffees?page=1&size=20&sort=price,desc`
 - 7. 關聯最佳化：
 - 將 `supId` 改為 `@ManyToOne Supplier supplier`，可同時提供展開/不展開參數 (? `expand=supplier`)
-

練習題與任務

1. 實作 `CoffeeDAO` 與 `CoffeeService` 完整 CRUD。
 2. 新增查詢：依 `supId` 查詢此供應商所有咖啡。
 3. 新增查詢：依價格區間查詢咖啡 (含排序)。
 4. 為 `Coffee` 加上 Bean Validation，測試 API 的錯誤回應。
 5. 撰寫 Postman/REST Client 測試腳本，覆蓋成功與錯誤情境。
-

小結

- 快速建立 `coffee` 模組的 DAO 與 REST Service。
- 注意 `Coffee` 的主鍵是字串 (`cofName`)，API 設計需相應調整。
- 透過驗證、關聯與統一錯誤處理，可提高系統的可用性與可維護性。